

Statera v3 — Vaults testnet à intégrer

2026-04-30

Statera v3 — Vaults testnet à intégrer

Date : 30 avril 2026 **Réseau** : Hyperliquid testnet **Chain ID** : 998
RPC : <https://rpc.hyperliquid-testnet.xyz/evm>

Infrastructure

	Adresse
Factory v3	0x57bc15d33db3e54de4040194916584247f9c685e
Implementation v3	0x0d161a069b1323d39a8afc2d4e202869ea1baa3b
Keeper EOA (owner)	0x1eE9C37E28D2DB4d8c35A94bB05C3f189191D506

Le keeper tourne en boucle 60s (PM2 process statera-v3-keeper)
et gère : activation Core, rebalance, settlement, emergency,
refresh d'indices HL.

Vaults déployés (3)

Vault 0 — HYPE / SOVY (équilibré)

Adresse	0xb9E15DC17a8133f0cdB778097D0169c2Ba284a77
Symbol	stHS0VY3
Counterpart token	SOVY (0x674d61f547AE1595f81369f7F37f7400C1210444)
Token index Core	1158
Spot market index	1080

Allocations cibles	48% HYPE / 48% SOVY / 4% USDC (par défaut)
État	Actif, 14.53 shares en circulation

Vault 1 — HYPE / UETH (preuve résilience HL)

Adresse	0x3e93bde3Aa75761AdB010088230f4d7C8F659a22
Symbol	UETH3
Counterpart token	UETH (0x5a1A1339ad9e52B7a4dF78452D5c18e8690746f3)
Token index Core	1242
Spot market index	1137
Allocations cibles	48% HYPE / 48% UETH / 4% USDC (par défaut)
État	Vide, utilisé pour la preuve on-chain du PROOF_REPORT

Vault 2 — HYPE / UNIT (asymétrique 30/70)

Adresse	0xcdcdc574d4f13f510ec2d12bcfd23003cb330f9a
Symbol	stHUNIT3
Counterpart token	UNIT (0x09f83c5052784c63603184e016e1db7a24626503)
Token index Core	1129
Spot market index	1054
Allocations cibles	30% HYPE / 70% UNIT / 0% USDC
État	Fraîchement déployé, vide, Core en cours d'activation par keeper

Lecture dynamique vs liste hardcodée

Si le frontend lit la liste depuis la factory (vaultCount() + allVaults(i)), rien à modifier — les 3 vaults sont déjà visibles.

Si la liste est hardcodée, ajouter le vault UNIT (0xcdcdc574d4f13f510ec2d12bcfd23003cb330f9a) à la config.

Transactions de référence

Action	Hash
Deploy vault UNIT	0x8a8bc03977bfe41557abb0554285a67c6e4677b0ca6d6a0f17
setTargetAllocations(3000, 7000, 0)	0xe99e48994a2883afc5462bc9d2b695450b8bee03a16ba173d2

Tester depuis le frontend

Pour chaque vault, le frontend doit pouvoir :

- `deposit()` payable — l'utilisateur envoie du HYPE natif
- `requestRedeem(uint256 shares)` — l'utilisateur demande un retrait
- `claimBatch(uint64 batchId)` — l'utilisateur récupère son HYPE après settlement
- Lectures : `sharePriceUsdc8()`, `grossAssets()`, `totalSupply()`, `balanceOf(user)`

L'ABI complet de `RebalancingVault` est dans `keeper/abi/RebalancingVault.json` ou récupérable via `forge inspect src/core/RebalancingVault.sol:RebalancingVault abi`.

Suggestion de tests pour les associés

1. **Vault SOVY** — déjà actif, `sharePrice` non-trivial (~\$1.90). Idéal pour tester le full cycle `deposit` → `requestRedeem` → `claim` sur un vault qui a déjà de la liquidité.
2. **Vault UETH** — vault vide, `sharePrice` = \$1.00 initial. Tester un premier `deposit` (premier user = ratio 1:1).
3. **Vault UNIT** — observer comment se comporte un vault avec allocations asymétriques (30% HYPE / 70% UNIT / 0% USDC). Le rebalance keeper devrait bridger massivement vers Core et tenir presque tout en UNIT entre cycles.